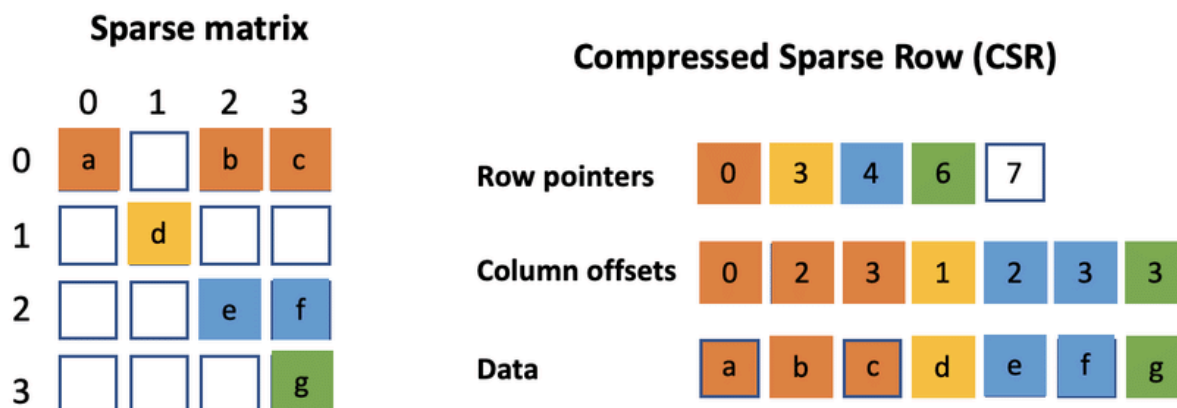


Assignment (SpMV)

When the matrices are sparse, it makes sense to efficiently store them in alternate representations than 2-D arrays. One such representation is Compressed Sparse Row (CSR) as shown below:



It stores the matrix as three 1-D arrays:

- **Data:** stores all non-zero elements in the row-wise order
- **Column offsets:** stores the column number of these non-zero elements
- **Row pointers array:** i^{th} element of this array stores the cumulative number of non-zero elements upto (not including) the i -th row. The first element is always zero, and hence it is of size $n+1$, where n is the number of rows in the matrix.

By putting subsequent nonzeros of the matrix rows in contiguous memory locations, the execution time is also reduced significantly for some computations.

Problem Statement

Write a program using CUDA for Sparse Matrix Vector Multiplication(SpMV), which multiplies a sparse matrix (in CSR format) with a dense Vector to generate a vector as output.

$$\begin{array}{c} \mathbf{A} \qquad \qquad \mathbf{B} \qquad \qquad \mathbf{C} \\ \left[\begin{array}{ccccccccc} 0 & 9 & 0 & 0 & 0 & 4 & 0 & 0 \\ 0 & 0 & 6 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 5 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 6 & 0 & 0 & 0 & 0 & 0 \end{array} \right]_{n \times n} \left[\begin{array}{c} 0 \\ 1 \\ 1 \\ 3 \\ 0 \end{array} \right]_{n \times 1} = \left[\begin{array}{c} 0 \\ 6 \\ 0 \\ 0 \\ 6 \end{array} \right]_{n \times 1} \end{array}$$

Input: i) File 1: sparse matrix in COO format

ii) File 2: A dense vector

iii) Number of threads (as input from user)

Output: Vector C

Instructions:

1. Read **inputfile.mtx**
 - a. Convert it into **CSR format** and store it as input **A**
 - b. Print #Rows, #Cols, #Non-Zeroes, #threads, and then the matrix.
2. Read **vector.txt** file as another input **B**.
3. Compute **A*B** sequentially and store output in C1 and print C1.
4. Compute **A*B** in parallel using CUDA and store output in C2, and print C2.
5. Compare the results of C1 and C2 to establish the correctness.
6. Vary the number of threads (1, 4, 16, 64 ,...) and compare the timings with the sequential code.
7. Report any other observations.